Tutorial #6 - Meteor Dash

Introduction

Welcome to the sixth AppGameKit Studio tutorial. Once again, these are aimed at coders who are just starting to explore the world of tier 1 (BASIC) coding.

We previously created some basic 2D games and have worked our way through a range of coding methods and techniques.

For this tutorial, we'll be delving into the world of 3D and learning how we can combine the skills we learnt from the previous lessons to blend 2D and 3D to create a game.

The entry level for these tutorials expects some initial understanding of the students in the use of AppGameKit Studio. For more beginner level tutorials they can read the AppGameKit Studio Online Guides and watch the YouTube Videos.

Students are also encouraged to join the Official AppGameKit Studio forums. There's a newbie section where new users will be helped by many experienced and friendly developers who use AppGameKit Studio all the time.

Objectives

These tutorials aim to teach coding and game development using a combination of examples and action-based teaching. You will learn from studying the code snippets, reading and understanding the comments and gain an additional understanding of each section through a number of simple exercises aimed at reinforcing each lesson and will help build your confidence by demonstrating the ease with which the code can be altered.

This sixth tutorial aims to expand on the concepts of basic coding covered by the previous tutorials and is designed to lay the groundwork of future developments. During this tutorial we will:

- Explore the use of 3D objects
- Combine 2D and 3D techniques
- Develop your own code for the first time to expand the game

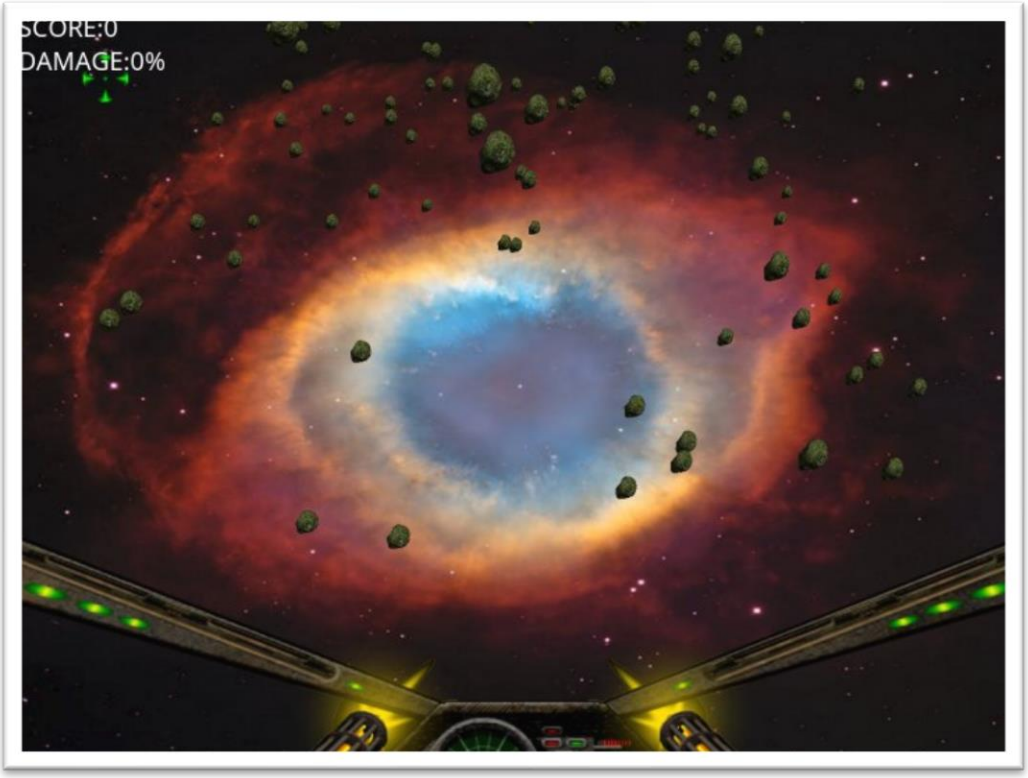
At any point, examples of almost all coding commands can be accessed from the IDE (integrated development environment) simply by either clicking directly to the left of the command and pressing F1 on the keyboard. You are encouraged to learn this method from the beginning as it will prove to be an invaluable reference guide and is a resource often used even by coders who are very familiar with AppGameKit Studio.

You can also access a full command reference from the IDE. By clicking Help on the top tool bar, you can gain access to a number of reference sources, including video tutorials, a list of useful keyboard shortcuts as well as the command reference.

Each section of code is available as a separate project which can be simply loaded into the IDE, but it's recommended that you learn to type the code into the IDE, as this will help to develop a familiarity with AppGameKit Studio overall and will promote confidence in using the functions of the IDE as well as reinforcing the learning process.

So, what game are we making?

For this tutorial, we will be making our first 3D game, Meteor Dash. Float through an endless asteroid field, blasting away at the drifting rocks to clear a path.



You will learn how we can combine slight variations of the code we've used for other games to blend 2D and 3D techniques. We will of course look at the fully documented code in detail and provide a number of exercises to expand your understanding.

Due to the nature of this game, we have another fairly short code block, so we'll post it all here and expand on the explanations where needed. Once again, apart from basic comments, we won't go into detail on areas we've already covered as we're confident that you've gained a good understanding of many of the techniques used from our previous tutorials, but as always, if you find anything confusing, don't be afraid to reference the earlier lessons.

Unlike our previous tutorials, we'll be challenging you to complete more complex coding exercises than before. Don't worry as everything you'll need has been covered in previous tutorials, so you'll have a point of reference.

As always, we set up the project and define the screen size.

```
// Project: Meteor Dash
// Created: 2017-07-18

// show all errors
SetErrorMode(2)

// set window properties
SetWindowTitle( "Meteor Dash" )
```

Unlike our previous projects, which have run in a window, we're running this game full screen.

The eagle eyed among you will notice that we've added a third number to SetWindowSize(1024,768,1). This 1 sets the application to run full screen, you can revert back to a windowed game by setting it to 0

```
SetWindowSize( 1024, 768, 1)
SetWindowAllowResize( 1 ) // allow the user to resize the window

// set display properties
SetVirtualResolution( 1024, 768 )
SetOrientationAllowed( 1, 1, 1, 1 )
SetSyncRate( 60, 0 )
UseNewDefaultFonts( 1 )
//set up basic 3D stuff
//set all 3D objects to appear at depth 50 is
//relation to 2D objects (text and sprites)
SetGlobal3DDepth(50)
//set the camera range
SetCameraRange(1,.1,10000)
//hide the mouse
SetRawMouseVisible(0)

//first we'll create a sprite for space
global space=1:LoadImage(space,"space.png")
CreateSprite(space,space)
//place it at the back, so it's drawn first
SetSpriteDepth(space,100)
//and another of the cockpit
Global cockpit=2
LoadImage(cockpit,"cockpit.png")
CreateSprite(cockpit,cockpit)
SetSpriteDepth(cockpit,10)
SetSpritePosition(cockpit,0,540)
//now we will need a crosshair
Global crosshair=3
LoadImage(crosshair,"crosshair.png")
CreateSprite(crosshair,crosshair)
```

Now let's create our very first 3D objects. We are only actually using one object for this game, and we'll use that as a reference to create clones of it later on in the code. For this tutorial we're using a .X format model, however, at the time of writing, AppGameKit Studio supports these formats: .X .3ds .md3 .smd .md5 .lwo. .ac .b3d .dae .3d .lws .ms3d .blend .m3 .obj and .ago

```
//let's create a 3D object.
Global rockparent=4:LoadObject(rockparent,"asteroid.x")
```

3D objects have an image assigned to them to give them more detail, this is referred to as a *texture*, but in all other aspects is no different from the images we would give to a sprite. So let's load in the image.

```
//we will need to give it a texture
LoadImage(rockparent,"asteroid.png")
```

This is where we assign the image. An object can have up to 8 (numbered 0 – 7) textures applied in stages for more detailed effects, such as adding depth or light to the images controlled via a special form of code called a shader. You don't need to worry about that for now as we'll only be using a single texture, so we'll apply it to stage 0.

```
//link the image with the object
SetObjectImage(rockparent,rockparent,0)
```

The following code should be very familiar to you if you've followed our previous tutorials. Even though we're positioning a 3D object rather than a 2D sprite, the only difference is that a 3D object has 3 coordinates rather than 2. These are X,Y,Z and can be thought of as Across, Up & Down, and Depth.

```
//hide the object by placing it a long way away
SetObjectPosition(rockparent,-10000,-10000,-10000)
//and make it invisible
SetObjectVisible(rockparent,0)
//we will also have some dust drifting in the background
Global dust=5
LoadImage(dust,"dust.png")
CreateSprite(dust,dust)
```

We have a new sprite command here, SetSpriteColorAlpha which simply allows us direct control of the alpha channel of a sprite, making it more or less transparent. This is the same as the 4th value in SetSpriteColor(red,green,blue,alpha), but means we can change the alpha without messing around with the colour.

```
SetSpriteColorAlpha(dust,80)
SetSpriteDepth(dust,99)
SetSpritePosition(dust,90,-180)
```

A large portion of the following code will be familiar, so we won't go into a great deal of depth apart from the comments. Don't forget that you can use F1 to get more detail on any of the commands if you need more information.

```
type rocks

        x as float
        y as float
        z as float
        scale as float
        object as integer
        angle as integer
        speed as integer
        distance as integer

endtype

Dim asteroids[100] as rocks

setupstorm()

//make some player stats

type player
        damage as integer
        score as integer
        recharge as integer
endtype

Global ship as player

ship.damage=0
ship.score=0

//taking a left from our previous tutorial
//we will be using 2D explosions, virtually
//duplicating the code from our Lunar Buggy
//tutorial

//now we will need some explosions
//the idea of explosions may seem complicated
//but are nothing more than an animated sprite
//played from start to finish
Type explosions
        x as float
        y as float
        sprite as integer
        active as integer
EndType

//we'll allow for 40
Dim bangit[40] as explosions

Global rockbang=6
```

```
loadimage(rockbang,"rockbang.png")

//and make the sprites
For makebang=1 to 40
        bangit[makebang].sprite=makebang+699
        createsprite(bangit[makebang].sprite,rockbang)
        //set the animation frame
        //the ship explosion is
        //64 sprite wide, 70 pixel hight and has 8 frames
        SetSpriteAnimation(bangit[makebang].sprite,64,70,8)
        SetSpritePosition(bangit[makebang].sprite,-10000,-10000)
        //same depth as the 3D objects
        SetSpriteDepth(bangit[makebang].sprite,50)
Next makebang

//load some sounds
Global laser:laser=LoadSound("laser.wav")
Global bang:bang=LoadSound("bang.wav")
Global shiphit:shiphit=LoadSound("shiphit.wav")
//timers
Global currenttime as integer
Global frameadjust as float
ship.recharge=currenttime

Do

        //set frame adjust to 60 * by the time it took to draw the last frame
        //this will produce a value we can multiple all movement by to ensure
        //the appearance of smooth movement
        frameadjust=60*GetFrameTime()
        currenttime=GetMilliseconds()

         showcrosshair()
        shootgun()
        moveasteroids()
        controlbangs()
        rotatedust()
        stats()

        If GetRawKeyState(27)
                end
        EndIf
    Sync()
Loop

Function showcrosshair()

        If currenttime-ship.recharge>500
                SetSpriteColor(crosshair,255,255,255,255)
                else
                SetSpriteColor(crosshair,127,0,0,255)
        Endif

        SetSpritePosition(crosshair,GetPointerX(),GetPointerY())

EndFunction

Function findundercrosshair()

//get the crosshair position on screen
        pointerx#=GetPointerX()+24
        pointery#=GetPointerY()+24

        //AppGameKit allows you to quickly
        //find a point in 3D space from a 2D position
        tempx#=Get3DVectorXFromScreen(pointerx#,pointery#)
        tempy#=Get3DVectorYFromScreen(pointerx#,pointery#)
        tempz#=Get3DVectorZFromScreen(pointerx#,pointery#)

        //we take those positions
        startx#=tempx#
        starty#=tempy#
        startz#=tempz#

        //calculate an end position
        endx#=10000*startx#
        endy#=10000*starty#
        endz#=10000*startz#
```

```
        //and raycast from the start position to the end position
        whichone=ObjectRayCast(0,startx#,starty#,startx#,endx#,endy#,endz#)

EndFunction whichone
```

OK, now it's time to make some changes, so let's start our first challenge.

EXERCISE: *Currently, the player can only shoot every 1/2 a second, or 500 milliseconds. Change the code to allow the player to firer faster or slower. You'll need to search the code as more than one change is needed to make this happen.*

```
Function shootgun()

whichone=findundercrosshair()

    if GetRawMouseLeftPressed() and currenttime-ship.recharge>500

        PlaySound(laser)

        ship.recharge=currenttime

        if whichone<>0
                tempdistance#=asteroids[whichone-99].distance
                makebang(GetPointerX()+24,GetPointerY()+24,tempdistance#)
                makenewasteroid(whichone-99)
                PlaySound(bang)
                inc ship.score,100
        endif
    endif

EndFunction
```

If the asteroid leaves the screen or is destroyed we'll need to move it back to the start so it can be reused. Once again, this code should be familiar to you.

```
Function makenewasteroid(which)

        //set up a new position
        tempx=random(1,1000)
        tempx=RandomSign(tempx)
        asteroids[which].x=tempx
        tempy=random(1,1000)
        asteroids[which].y=-tempy
        //at a random distance
        asteroids[which].z=random(1000,4000)
        asteroids[which].angle=random(1,360)
        asteroids[which].speed=random(1,3)
        //set the objects position
        SetObjectPosition(asteroids[which].object,asteroids[which].x,asteroids[which].y,aster
oids[which].z)

EndFunction
```

We've got some space dust drifting around in the background, we rotate it to give the impression of movement.

```
Function rotatedust()

        //let's find the angle of the dust sprite
        tempdust=GetSpriteAngle(dust)
        //increase it
        inc tempdust,1*frameadjust
        //set the new angle
        SetSpriteAngle(dust,tempdust)

EndFunction
```

Now let's move those asteroids around. We're only interested in the Z movement for this game, which can be thought of as the depth. (Because the asteroids are moving towards the camera.).

```
Function moveasteroids()

        //run through all the sprites
        For move=1 to 100
        //move the Z position so the asteriods
        //move towards the camera
        Dec asteroids[move].z,asteroids[move].speed*frameadjust
        //set the new position

        SetObjectPosition(asteroids[move].object,asteroids[move].x,asteroids[move].y,asteroid
s[move].z)
```

Just like sprites, 3D objects can be rotated. Now sprites can only be rotated left and right, but 3D objects obviously have 3 dimensions, and so can be rotated in the X, Y and Z axes. For this tutorial, we've rotate the object on the X and Z to give it some animation.

```
//rotate the object by the speed of the asteroid
tempanglex=GetObjectAngleX(asteroids[move].object)+asteroids[move].speed*frameadjust
tempanglez=GetObjectAngleZ(asteroids[move].object)+asteroids[move].speed*frameadjust
//set the new rotation
        SetObjectRotation(asteroids[move].object,tempanglex,GetObjectAngleY(asteroids[move].o
bject),tempanglez)

//if the asteroid is behind the player camera
//reset it so we have endless asteroids
        if asteroids[move].z<-100
                makenewasteroid(move)
        endif

//get the asteroids distance

asteroids[move].distance=distance(0,0,0,asteroids[move].x,asteroids[move].y,asteroids[move].z
)
```

If the asteroid is within 400 units of the camera (player position in this case), we can assume that it has collided with the player ship. This kind of trick is very performance friendly, and distance collision can be used in many cases where more complex collision isn't needed.

```
//if the asteroid is within 400 units and isn't behind the camera
//assume it has hit the player
If asteroids[move].distance<400 and asteroids[move].distance>0
```

Now for a clever bit! We need to find a position in 2D space from the position of the 3D object. Basically, where on the screen is it. Luckily for us, AppGameKit Studio has commands for this. (It also has commands to translate 2D screen positions into 3D space, as we'll see later).

```
//find a point in 2D space from a 3D object
        x1#=GetScreenXFrom3D(asteroids[move].x,asteroids[move].y,asteroids[move].z)
        Y1#=GetScreenXFrom3D(asteroids[move].x,asteroids[move].y,asteroids[move].z)
        //make an explosion at the screen position
        makebang(x1#,y1#,asteroids[move].distance)
        //destroy the asteroid simply by reseting it's position
        makenewasteroid(move)
        //increase ship damage randomly
        inc ship.damage,random(4,10)
        //play the ship hit and explosion sounds
        PlaySound(shiphit)
        PlaySound(bang)

        //don't let ship damage exceed 100%
        if ship.damage>100
                ship.damage=100
        endif
        endif
        //check the next asteroid
        Next move

EndFunction
```

```
//make the storm
Function setupstorm()

        For makerocks=1 to 100
        //assign the object a number
        asteroids[makerocks].object=99+makerocks

        //if the object doesn't exisit
        //make a copy of it
        If GetObjectExists(asteroids[makerocks].object)=0
        CloneObject(asteroids[makerocks].object,rockparent)
        SetObjectVisible(asteroids[makerocks].object,1)
        endif

        //randomly set the objects position
        //and give it a random speed and angle
        tempx=random(1,1000)
        tempx=RandomSign(tempx)
        asteroids[makerocks].x=tempx
        tempy=random(1,1000)
        asteroids[makerocks].y=-tempy
        asteroids[makerocks].z=random(1000,4000)

        asteroids[makerocks].angle=random(1,360)
        asteroids[makerocks].speed=random(1,3)

        //set the objects position in 3D space
        SetObjectPosition(asteroids[makerocks].object,asteroids[makerocks].x,asteroids[makero
cks].y,asteroids[makerocks].z)

Next makerocks

EndFunction
```

This is where we display our game stats. Not very inspiring we're sure you'll agree. Which is convenient, as this leads us neatly into our next challenge.

EXERCISE: *Create a text object to replace the Print commands to display the Score and the Ship's damage. Don't forget you'll need to set the string as well.*

EXERCISE: *Create a simple sprite to display the ship's damage. We've provided an image called healthbar for you, but you'll need to add the code to load the image, create the sprite. There are several methods of creating this kind of indicator, but we recommend taking a look at SetSpriteSize for clues.*

```
//display stats
Function stats()

        Print("SCORE:"+str(ship.score))
        Print("DAMAGE:"+str(ship.damage)+"%")

EndFunction
```

This very useful little function returns an approximate distance between 2 points in 3D space. It's fast and accurate enough for most gaming purposes so feel free to use it in your own projects. We aren't aiming to teach math for these tutorials, but for reference the commands SQRT returns the square root of a float value.

```
//find the distance between two points
//in 3D space
Function distance(x1#,y1#,z1#,x2#,y2#,z2#)

   returndistance#=0

   distx#=x1#-x2#
   disty#=y1#-y2#
   distz#=z1#-z2#

   returndistance#=Sqrt(distx#*distx#+disty#*disty#+distz#*distz#)
```

```
EndFunction returndistance#
```

This is where we start to re-use some of our functions from our previous tutorials. Hopefully this will encourage you to start creating your own library of useful code.

```
//make an explosion
Function makebang(x,y,size#)

        if size#>1000 then size#=1000
        size#=1000-size#

        if size#<16 then size#=16

        //find a free space
        usethis=findfreebang()

        //if it exists
        If usethis<>0

        //set the sprites position
        bangit[usethis].x=x-(size#/2)
        bangit[usethis].y=y-(size#/2)
        //make it active
        bangit[usethis].active=1
        //set the sprite playing at 12 frames a second
        PlaySprite(bangit[usethis].sprite,12)

        SetSpriteSize(bangit[usethis].sprite,size#,size#)

        Endif

EndFunction

Function findfreebang()

        //find a free explosion
        //this is exactly like the code for finding a player fire space
        //and an alien fire space
        use=0

        For find=1 to 40
                if bangit[find].active=0
                        use=find
                        find=41
                Endif
        Next find

EndFunction use

Function controlbangs()

        //now we will take a look at the explosion sprites
        For check=1 to 40
                //as always, if it active
                If bangit[check].active=1
                //move is slightly to the left to match the lands movement
                bangit[check].x=bangit[check].x-1*frameadjust
                //we can use GetSpriteCurrentFrame to find out the current frame it's using
                //we can use GetSpriteFrameCount to find out how many frames it has
                //if GetSpriteCurrentFrame()=GetSpriteFrameCount()
                //we know the sprite has played all of it's frames
                If
GetSpriteCurrentFrame(bangit[check].sprite)=GetSpriteFrameCount(bangit[check].sprite)
                //so we make it inactive (and available again)
                //and move it off screen completely
                bangit[check].active=0
                bangit[check].x=-10000
                bangit[check].y=-10000
                EndIf
                //update the sprite position
                SetSpritePosition(bangit[check].sprite,bangit[check].x,bangit[check].y)
                EndIf
                //keeep on checking
        Next check
EndFunction
```

That's it for now, all that remains is to leave you with a couple more challenges.

EXERCISE: *Some of our previous games had a main menu. Create a mainmenu() function that starts the game and is returned to once the player ship has been destroyed (damage is at 100%).*

For this exercise you should:

1. *Display a game title*
2. *Give instructions on which key to press to start the game and allow that key to start the game.*
3. *Before returning to the menu the game must report failure and allow enough time for the player to see the report*

EXERCISE: *Our game clearly needs some music, but we've left it to you. Find yourself some music and add it to the game. If you're struggling for any, you can find some in our Lunar Buggy tutorial if needed. The game should loop the music. There are several ways you can do this, study earlier tutorials if you need to or read the command reference for PlayMusic and PlayMusicOgg for clues.*